

Technische Dokumentation

Backend der ILSE-Plattform

Überblick über Klassen, Aufbau und Zusammenspiel der Backend-Komponenten

1. Einleitung

Dieses Dokument beschreibt den Aufbau des Backends der ILSE-Plattform, einer Lernanwendung mit Themen, Räumen, Aufgaben, Levels und Erfahrungspunkten. Schülerinnen und Schüler melden sich über einen Einmal-Code (OTP) an und bearbeiten Aufgaben in verschiedenen Räumen, um Erfahrungspunkte und neue Level freizuschalten.

Die Dokumentation richtet sich an alle Projektbeteiligten, die einen Überblick über die vorhandenen Klassen benötigen, unabhängig vom technischen Hintergrund. Jede Klasse wird einheitlich beschrieben: Zweck, enthaltene Daten bzw. Methoden und Zusammenhang mit anderen Klassen.

1.1 Aufbau des Backends

Der Code ist in vier Schichten aufgeteilt:

- Domain – enthält die fachlichen Objekte (z. B. Benutzer, Raum, Aufgabe) und die reinen Geschäftsregeln. Diese Schicht kennt keine Datenbank und keine Web-Technik.
- Application – enthält die Services, die einen konkreten Ablauf steuern (z. B. "Benutzer einloggen"), sowie die DTOs für den Datenaustausch mit der Außenwelt.
- Persistence – kümmert sich um die Datenbank: den Datenbankkontext, die Repositories (Lesen/Schreiben) und die Zuordnung der Domain-Objekte auf Tabellen.
- API (ILSE_Plattform) – die Controller, die HTTP-Anfragen entgegennehmen und an die passenden Services weiterleiten, sowie der Startpunkt Program.cs.

Eine Anfrage läuft also typischerweise so durch das System: Controller nimmt die Anfrage entgegen → ruft einen Service auf → der Service nutzt ein oder mehrere Repositories → das Repository liest oder schreibt über den Datenbankkontext (AppDbContext) in der Datenbank.

1.2 Beschreibungsschema

Jede Klasse wird, abhängig von ihrer Art, nach folgendem einheitlichen Schema beschrieben:

- Fachklassen (Domain-Objekte): Zweck, Attribute-Tabelle, wichtige Methoden.
- DTOs (reine Datenpakete für Anfragen/Antworten): Zweck, Felder-Tabelle.
- Services, Repositories und Hilfsklassen: Zweck, Abhängigkeiten, Methoden-Tabelle.
- Controller: Basis-Route, Zweck, Tabelle der Endpunkte.

Wo im Code etwas unfertig, uneindeutig oder auffällig ist, wird das als kurzer Hinweis direkt bei der jeweiligen Klasse oder gesammelt im Anhang vermerkt.

2. Fachklassen (Domain-Objekte)

Diese Klassen bilden die fachliche Logik der Plattform ab. Anstelle öffentlicher Konstruktoren stehen die Methoden CreateNew (neues Objekt anlegen) und Reconstruct (bestehendes Objekt aus der Datenbank wiederherstellen) zur Verfügung. Attribute sind von außen nur lesbar; Änderungen erfolgen ausschließlich über eigene Methoden, um ungültige Zustände zu verhindern.

2.1 Benutzer und Zugang

User

Zweck: Repräsentiert eine Person, die die Plattform nutzt – Schüler/in, Lehrer/in oder Admin – zusammen mit ihrer Rolle und ihren gesammelten Erfahrungspunkten.

Attribut	Typ	Bedeutung
Id	Guid	Eindeutige Kennung des Benutzers
Username	string	Angezeigter Benutzername
Role	UserRole	Rolle: Student, Teacher oder Admin
ExperiencePoints	int	Gesammelte Erfahrungspunkte (XP)

Wichtige Methoden

- CreateNew(username, role) – legt einen neuen Benutzer mit 0 XP an
- Reconstruct(...) – baut einen bestehenden Benutzer aus gespeicherten Daten wieder auf
- AddExperience(points) – erhöht die Erfahrungspunkte um den angegebenen Wert

UserRole (Enum)

Zweck: Legt die drei möglichen Rollen fest, die ein Benutzer haben kann.

Attribut	Typ	Bedeutung
Student	-	Lernende/r, kann Aufgaben bearbeiten
Teacher	-	Kann Einmal-Codes für Schüler erstellen
Admin	-	Höchste Berechtigungsstufe

UserRoleHierarchy

Zweck: Ordnet jeder Rolle einen Rang zu (Student = 0, Teacher = 10, Admin = 20) und stellt darüber eine einfache Berechtigungsprüfung bereit.

Wichtige Methoden

- IsAtLeast(minimum) – prüft, ob eine Rolle mindestens den Rang einer Mindestrolle hat, z. B. role.IsAtLeast(UserRole.Teacher)

UserCredential

Zweck: Speichert die Zugangsdaten eines Benutzers getrennt vom User-Objekt: einen gehashten geheimen Zugangscode (SecretId) und ein gehashtes Passwort.

Attribut	Typ	Bedeutung
Id	Guid	Eindeutige Kennung
UserId	Guid	Verweist auf den zugehörigen Benutzer
SecretIdHash	string	Gehashte geheime Zugangs-ID
PasswordHash	string	Gehashtes Passwort
PasswordSalt	string	Salt für die Passwort-Hash-Berechnung
CreatedAt	DateTime	Erstellungszeitpunkt

Wichtige Methoden

- CreateNew(...) – legt neue Zugangsdaten an, Zeitstempel wird automatisch gesetzt
- Reconstruct(...) – stellt bestehende Zugangsdaten wieder her
- UpdatePassword(hash, salt) – ersetzt Passwort-Hash und Salt

AppUser

Zweck: Erweitert den Standard-Benutzer von ASP.NET Core Identity (IdentityUser<Guid>) um zwei zusätzliche Felder. Diese Klasse wird parallel zu User für die technische Anmeldeverwaltung (Identity-Framework) genutzt.

Attribut	Typ	Bedeutung
CreatedAt	DateTime	Erstellungszeitpunkt, wird automatisch gesetzt
LastLoginAt	DateTime?	Zeitpunkt der letzten Anmeldung

Hinweis: User (Domain) und AppUser (Identity) sind zwei getrennte Klassen für "einen Benutzer" – User trägt die fachlichen Daten, AppUser die technischen Anmeldedaten des Identity-Frameworks.

OneTimePassword

Zweck: Bildet einen Einmal-Code (OTP) ab, mit dem sich ein Schüler oder eine Schülerin einmalig anmelden kann. Der Code wird nur gehasht gespeichert und ist zeitlich begrenzt gültig.

Attribut	Typ	Bedeutung
Id	Guid	Eindeutige Kennung
CodeHash	string	Gehashter Code (Klartext wird nicht gespeichert)
TargetRole	UserRole	Rolle, für die der Code ausgestellt wurde
IsUsed	bool	Ob der Code bereits eingelöst wurde
CreatedAt	DateTime	Erstellungszeitpunkt
ExpiresAt	DateTime	Ablaufzeitpunkt
CreatedByUserId	Guid	Wer den Code erstellt hat (Lehrer/Admin)
UserId	Guid	Für welchen Benutzer der Code gilt

Wichtige Methoden

- CreateNew(...) – erzeugt einen neuen Code mit Ablaufdatum (heute + Gültigkeitstage)
- Reconstruct(...) – stellt einen bestehenden Code wieder her
- IsExpired() – prüft, ob der Code abgelaufen ist
- MarkAsUsed() – markiert den Code als verbraucht; wirft einen Fehler, wenn er bereits benutzt oder abgelaufen ist

2.2 Lerninhalte: Themen, Räume, Level

Topic

Zweck: Ein Thema, das mehrere Räume zusammenfasst (z. B. ein Lernbereich).

Attribut	Typ	Bedeutung
Id	Guid	Eindeutige Kennung
Name	string	Name des Themas
Rooms	IReadOnlyList<Room>	Räume, die zu diesem Thema gehören

Wichtige Methoden

- CreateNew(name) – legt ein neues Thema ohne Räume an
- Reconstruct(...) – stellt ein bestehendes Thema mit seinen Räumen wieder her
- AddRoom(room) – fügt einen Raum hinzu (keine doppelten Einträge)
- RemoveRoom(room) – entfernt einen Raum

Room

Zweck: Ein Raum innerhalb eines Themas, der eine Sammlung von Aufgaben enthält und ab einem bestimmten Level zugänglich ist.

Attribut	Typ	Bedeutung
Id	Guid	Eindeutige Kennung
Name	string	Name des Raums
UnlockLevel	int	Ab welchem Level der Raum zugänglich ist
CompletionExperiencePoi	int	XP-Belohnung für das Abschließen des Raums

Attribut	Typ	Bedeutung
nts		
Exercises	IReadOnlyList<Exercise>	Aufgaben in diesem Raum

Wichtige Methoden

- CreateNew(name, unlockLevel, completionExperiencePoints) – legt einen neuen, leeren Raum an
- Reconstruct(...) – stellt einen bestehenden Raum mit seinen Aufgaben wieder her

Level

Zweck: Definiert eine Levelstufe: einen Namen, ein Symbolbild und die dafür nötigen Erfahrungspunkte.

Attribut	Typ	Bedeutung
Id	Guid	Eindeutige Kennung
Name	string	Bezeichnung des Levels
ToolImage	string	Pfad/Verweis auf ein Icon-Bild für dieses Level
ExperiencePoints	int	Ab wie viel XP dieses Level erreicht ist

Wichtige Methoden

- CreateNew(name, toolImage, experiencePoints) – legt ein neues Level an
- Reconstruct(...) – stellt ein bestehendes Level wieder her

2.3 Aufgaben (Exercises)

Es gibt eine gemeinsame Basisklasse Exercise und darauf aufbauend vier konkrete Aufgabentypen. Jeder Typ speichert zusätzlich zu den Basisdaten die für ihn nötigen Zusatzinformationen (z. B. Antwortoptionen oder Klickbereiche).

Exercise (abstrakte Basisklasse)

Zweck: Gemeinsame Grundlage aller Aufgabentypen. Legt fest, welche Daten jede Aufgabe mindestens hat.

Attribut	Typ	Bedeutung
Id	Guid	Eindeutige Kennung
Title	string	Titel der Aufgabe
Description	string	Beschreibungstext
ExperiencePoints	int	XP-Belohnung für das Lösen der Aufgabe

Wichtige Methoden

- ValidateAnswer(answer) – abstrakte Methode, die jeder Aufgabentyp umsetzen muss, um eine Antwort zu prüfen

Hinweis: ValidateAnswer ist in allen vier abgeleiteten Klassen bisher nur als Platzhalter vorhanden (wirft NotImplementedException); die Bewertungslogik ist noch nicht implementiert.

MultipleChoiceMultiAnswerExercise

Zweck: Multiple-Choice-Aufgabe mit mehreren möglichen Antwortoptionen, bei der auch mehrere Optionen richtig sein können.

Attribut	Typ	Bedeutung
Options	IReadOnlyList<MCOption>	Alle wählbaren Antwortoptionen
CorrectOptionIds	IReadOnlyList<MCOption>	Die als richtig markierten Optionen

DragAndDropExercise

Zweck: Aufgabe, bei der Elemente per Drag-and-Drop den richtigen Ablagezonen zugeordnet werden müssen.

Attribut	Typ	Bedeutung
Items	IReadOnlyList<DraggableItem>	Ziehbare Elemente

Attribut	Typ	Bedeutung
Zones	ReadOnlyList<DropZone>	Verfügbare Ablagezonen
CorrectMappings	ReadOnlyList<DragAndDropMapping>	Richtige Zuordnung Element → Zone

LinkingExercise

Zweck: Zuordnungsaufgabe, bei der linke und rechte Einträge miteinander verbunden werden müssen.

Attribut	Typ	Bedeutung
LeftItems	ReadOnlyList<Link>	Einträge auf der linken Seite
RightItems	ReadOnlyList<Link>	Einträge auf der rechten Seite
CorrectLinks	ReadOnlyList<Link>	Die richtigen Verbindungen

ClickableImageExercise

Zweck: Aufgabe, bei der auf bestimmte Bereiche innerhalb eines Bildes geklickt werden muss.

Attribut	Typ	Bedeutung
ImageUrl	string	Pfad/Verweis auf das anzuzeigende Bild
ClickableAreas	ReadOnlyList<ClickableArea>	Anklickbare Bereiche im Bild

OrderExercise

Zweck: Fünfter Aufgabentyp, vermutlich für Sortieraufgaben vorgesehen.

Hinweis: Die Klasse ist aktuell leer und enthält weder Attribute noch Methoden – dieser Aufgabentyp ist noch nicht umgesetzt.

Hilfsklassen für Aufgaben

Diese kleinen Klassen liefern die Bausteine, aus denen die Aufgabentypen oben zusammengesetzt sind. Sie speichern jeweils nur wenige Daten und werden über CreateNew / Reconstruct erzeugt.

Klasse	Attribute	Verwendung
ClickableArea	X, Y, Width, Height (int)	Rechteckiger Klickbereich in ClickableImageExercise
DraggableItem	Content (string)	Ein ziehbares Element in DragAndDropExercise
DropZone	Label (string)	Eine Ablagezone in DragAndDropExercise
DragAndDropMapping	ItemId, DropZoneId (Guid)	Richtige Zuordnung Element → Zone
Link	LeftItem, RightItem (string)	Ein Paar in LinkingExercise
MCOption	OptionText (string)	Eine Antwortoption in MultipleChoiceMultiAnswerExercise

2.4 Fortschritt (Progress)

UserProgress

Zweck: Hält den allgemeinen Fortschritt eines Benutzers in Form von Erfahrungspunkten fest.

Attribut	Typ	Bedeutung
Id	Guid	Eindeutige Kennung
UserId	Guid	Zugehöriger Benutzer
ExperiencePoints	int	Gesammelte Erfahrungspunkte

Wichtige Methoden

- CreateNew(userId) – legt einen neuen Fortschrittseintrag mit 0 XP an
- Reconstruct(...) – stellt einen bestehenden Eintrag wieder her

ExerciseProgress

Zweck: Hält fest, welche Aufgaben ein Benutzer bereits abgeschlossen hat.

Attribut	Typ	Bedeutung
Id	Guid	Eindeutige Kennung
UserId	Guid	Zugehöriger Benutzer
Exerciseld	Guid	Betrachtete Aufgabe
CompletedExercises	List<Guid>	Liste bereits abgeschlossener Aufgaben

Wichtige Methoden

- CreateNew(userId, exerciseld) – legt einen neuen, leeren Fortschrittseintrag an
- Reconstruct(...) – stellt einen bestehenden Eintrag wieder her
- MarkAsCompleted() – trägt die Aufgabe als abgeschlossen ein (keine doppelten Einträge)

RoomProgress

Zweck: Hält fest, welche Räume ein Benutzer bereits abgeschlossen hat.

Attribut	Typ	Bedeutung
UserId	Guid	Zugehöriger Benutzer
RoomId	Guid	Betrachteter Raum
CompletedRooms	List<Guid>	Liste bereits abgeschlossener Räume

Hinweis: Die Klassendatei war in diesem Export nicht enthalten; der Aufbau ergibt sich aus der Datenbank-Konfiguration und dem zugehörigen Repository.

3. Datenübertragungsobjekte (DTOs)

DTOs ("Data Transfer Objects") sind einfache Datenpakete ohne eigene Fachlogik. Sie legen fest, welche Daten eine Anfrage an die API enthalten muss (Request) und welche Daten als Antwort zurückkommen (Response).

3.1 Anmeldung und Einmal-Codes

Im Code existieren für diesen Bereich zwei Varianten nebeneinander, die zu zwei unterschiedlichen Anmelde-Konzepten gehören:

Variante A – Anmeldung nur mit Code (Application.DTOs, AuthDTOs.cs)

CreateOtpRequest

Zweck: Wird von Lehrer/Admin gesendet, um einen neuen Code für eine Zielrolle zu erstellen.

Feld	Typ	Bedeutung
TargetRole	UserRole	Rolle, für die der Code gilt

CreateOtpResponse

Zweck: Antwort nach dem Erstellen eines Codes.

Feld	Typ	Bedeutung
Code	string	Der erzeugte Klartext-Code
Username	string	Automatisch vergebener Benutzername

LoginRequest

Zweck: Wird gesendet, um sich mit dem Einmal-Code anzumelden.

Feld	Typ	Bedeutung
Code	string	Der erhaltene Einmal-Code

LoginResponse

Zweck: Antwort nach erfolgreicher Anmeldung.

Feld	Typ	Bedeutung
Token	string	JWT-Zugangstoken
Username	string	Benutzername
Role	string	Rolle des Benutzers

Variante B – Anmeldung mit geheimer ID, Passwort und Code (eigene Dateien, kein gemeinsamer Namespace)

CreateOtpRequest (Variante B)

Zweck: Wird gesendet, um für einen bestehenden Benutzer (per UserId) einen Code zu erzeugen.

Feld	Typ	Bedeutung
UserId	Guid	Betroffener Benutzer

CreateOtpResponse (Variante B)

Zweck: Antwort nach dem Erstellen eines Codes, inklusive Ablaufzeitpunkt.

Feld	Typ	Bedeutung
Code	string	Erzeugter Code
Username	string	Benutzername
ExpiresAt	DateTime	Ablaufzeitpunkt des Codes

LoginRequest (Variante B)

Zweck: Wird für die Anmeldung mit geheimer ID, Passwort und Code gesendet.

Feld	Typ	Bedeutung
SecretId	string	Geheime Zugangs-ID
Password	string	Passwort
OtpCode	string	Zusätzlicher Einmal-Code

LoginResponse (Variante B)

Zweck: Antwort nach erfolgreicher Anmeldung.

Feld	Typ	Bedeutung
Token	string	JWT-Zugangstoken

SetCredentialRequest

Zweck: Wird gesendet, um für einen Benutzer erstmalig ein Passwort zu vergeben.

Feld	Typ	Bedeutung
UserId	Guid	Betroffener Benutzer
Password	string	Neues Passwort

SetCredentialResponse

Zweck: Antwort nach dem Setzen des Passworts – enthält die geheime ID einmalig im Klartext.

Feld	Typ	Bedeutung
SecretId	string	Geheime Zugangs-ID, wird nur hier einmalig im Klartext ausgegeben

Hinweis: Beide Varianten sind gleichzeitig im Code vorhanden. Beim Weiterentwickeln sollte geklärt werden, welche der beiden Anmelde-Konzepte tatsächlich verwendet werden soll, um doppelte, gleichnamige Klassen zu vermeiden.

3.2 Themen und Räume

TopicResponse

Zweck: Antwort mit den Daten eines Themas.

Feld	Typ	Bedeutung
Id	Guid	Eindeutige Kennung
Name	string	Name des Themas

TopicRequest

Zweck: Anfrage zum Anlegen eines neuen Themas. Existiert als record (Application.DTOs) und zusätzlich als fast identische Klasse in ILSE_Plattform.RequestObjects.TopicRequests.

Feld	Typ	Bedeutung
Name	string	Name des neuen Themas

RoomResponse

Zweck: Antwort mit den Daten eines Raums.

Feld	Typ	Bedeutung
Id	Guid	Eindeutige Kennung
Name	string	Name des Raums
UnlockLevel	int	Ab welchem Level zugänglich
CompletionExperiencePoints	int	XP-Belohnung für Abschluss

3.3 Aufgaben

ExerciseResponse ist die gemeinsame Basis für die Antwort auf eine Aufgabe. Je nach Aufgabentyp wird eine der vier abgeleiteten Antwortklassen verwendet, erkennbar am Feld Type.

DTO	Zusätzliche Felder	Type-Wert
ExerciseResponse (Basis)	Id, Title, Description, ExperiencePoints, Type	-
MultipleChoiceResponse	Options (List<string>)	"MultipleChoice"
DragAndDropResponse	Items, Zones (je List<string>)	"DragAndDrop"
LinkingResponse	LeftItems, RightItems (je List<string>)	"Linking"
ClickableImageResponse	ImageUrl (string)	"ClickableImage"

3.4 Einstellungen

OtpSettings

Zweck: Konfigurationswerte für die Erstellung von Einmal-Codes, werden aus der appsettings-Datei gelesen.

Feld	Typ	Bedeutung
ExpiryDays	int	Gültigkeitsdauer eines Codes in Tagen (Standard: 3)
CodeLength	int	Länge des erzeugten Codes (Standard: 8)

4. Services (Geschäftslogik)

Services setzen einen konkreten Ablauf um, meist im Auftrag eines Controllers. Sie enthalten selbst keine Datenbankzugriffe, sondern nutzen dafür Repositories.

4.1 Anmeldung und Codes

AuthService

Zweck: Meldet einen Schüler oder eine Schülerin per Einmal-Code an.

Abhängigkeiten: IOTPRepository, IUserRepository, JwtTokenService

Methode	Beschreibung
LoginWithOtpAsync(request)	Sucht den Code, prüft ob er bereits benutzt wurde, lädt den zugehörigen Benutzer, markiert den Code als verbraucht und stellt ein JWT-Token aus.

Hinweis: Die Schnittstelle IAuthService sieht zusätzlich eine Methode SetCredentialsAsync vor, die in dieser Klasse aktuell nicht umgesetzt ist.

JwtTokenService

Zweck: Erzeugt signierte Zugangstoken (JWT) für angemeldete Benutzer.

Abhängigkeiten: IConfiguration (liest Jwt:Secret, Jwt:Issuer, Jwt:Audience, Jwt:ExpiryMinutes)

Methode	Beschreibung
GenerateToken(user)	Erstellt ein JWT mit Benutzer-Id, Name und Rolle als Claims, signiert mit HMAC-SHA256. Standardgültigkeit: 480 Minuten.

OtpService

Zweck: Erstellt neue Einmal-Codes für Schüler und Schülerinnen.

Abhängigkeiten: IOTPRepository, IUserRepository, OtpSettings

Methode	Beschreibung
CreateOtpAsync(request, callerId)	Lädt den Zielbenutzer, erzeugt einen zufälligen Code, speichert nur dessen Hash und gibt den Klartext-Code einmalig zurück.

4.2 Lerninhalte

LevelService

Zweck: Liefert die vorhandenen Level und ermittelt das aktuelle Level zu einem XP-Stand.

Abhängigkeiten: ILevelRepository

Methode	Beschreibung
GetAllAsync()	Gibt alle Level zurück.
GetCurrentLevelAsync(experiencePoints)	Wählt das höchste Level, dessen Schwellenwert bereits erreicht wurde.

TopicService

Zweck: Stellt Lese- und Anlegefunktionen für Themen bereit.

Abhängigkeiten: ITopicRepository

Methode	Beschreibung
GetAllAsync()	Gibt alle Themen zurück.
GetByIdAsync(id)	Gibt ein einzelnes Thema zurück.
CreateAsync(request)	Legt ein neues Thema an und speichert es.

RoomService

Zweck: Stellt Lesefunktionen für Räume bereit.

Abhängigkeiten: IRoomRepository

Methode	Beschreibung
GetAllByTopicIdAsync(topicId)	Gibt alle Räume eines Themas zurück.
GetByIdAsync(roomId)	Gibt einen einzelnen Raum zurück.

ExerciseService

Zweck: Stellt Lesefunktionen für Aufgaben bereit.

Abhängigkeiten: IExerciseRepository

Methode	Beschreibung
GetExercisesByRoomAsync(roomId)	Gibt alle Aufgaben eines Raums zurück.
GetByIdAsync(exerciseId)	Gibt eine einzelne Aufgabe zurück.

4.3 Fortschrittsberechnung

ProgressService (Domain.Services)

Zweck: Reine Hilfsberechnungen rund um Fortschritt, unabhängig von einer konkreten Entität – ob ein Nutzer alle geforderten Aufgaben oder Räume abgeschlossen hat und wie viel Prozent bereits erledigt sind.

Methode	Beschreibung
IsCompleted(required, completed)	Prüft, ob alle geforderten IDs in der Liste der abgeschlossenen IDs enthalten sind.
GetCompletionPercentage(required, completed)	Berechnet den Fortschritt in Prozent (0–100); ergibt 100, wenn nichts gefordert ist.

Hinweis: Diese Klasse implementiert nicht die Schnittstelle IProgressService aus der Application-Schicht (GetUserProgressAsync, MarkExerciseCompletedAsync usw.). Eine Klasse, die IProgressService umsetzt, war in diesem Export nicht enthalten.

5. Repositories (Datenzugriff)

Repositories kapseln den Zugriff auf die Datenbank. Jedes fachliche Objekt hat ein eigenes Repository, das von einer gemeinsamen Basisklasse erbt und um spezielle Abfragen ergänzt wird.

BaseRepository<T> / IBaseRepository<T>

Zweck: Generische Basisklasse mit den Standard-Datenbankoperationen, die jedes Repository benötigt. Arbeitet direkt über den DbContext.

Methode	Beschreibung
AddAsync(obj)	Fügt ein neues Objekt hinzu und speichert.
UpdateAsync(obj)	Aktualisiert ein bestehendes Objekt und speichert.
DeleteAsync(id)	Löscht ein Objekt anhand seiner Id.
GetByIdAsync(id)	Lädt ein einzelnes Objekt anhand seiner Id.
GetAllAsync()	Lädt alle Objekte dieses Typs.

Hinweis: In DeleteAsync ist die Prüfung vertauscht: Der Fehler "Entity with id ... not found" wird aktuell ausgelöst, wenn der Eintrag gefunden wurde, statt wenn er fehlt. Das sollte vor dem produktiven Einsatz korrigiert werden.

5.1 Übersicht der spezialisierten Repositories

Jedes der folgenden Repositories erbt die fünf Standardmethoden von BaseRepository und ergänzt sie um die unten aufgeführten, fachspezifischen Abfragen.

UserRepository

Zweck: Zugriff auf Benutzer, inklusive Abgleich mit dem Identity-System für E-Mail-Suche.

Methode	Beschreibung
GetByUsernameAsync(username)	Sucht einen Benutzer über seinen Benutzernamen.
GetByEmailAsync(email)	Sucht zunächst den zugehörigen AppUser über den UserManager, dann den passenden Domain-Benutzer.
GetUsersByRoleAsync(role)	Gibt alle Benutzer mit einer bestimmten Rolle zurück.
GetExperiencePointsAsync(userId)	Liest den aktuellen XP-Stand eines Benutzers.
UpdateExperiencePointsAsync(userId, xp)	Erhöht die XP eines Benutzers und speichert.

UserCredentialRepository

Zweck: Zugriff auf gespeicherte Zugangsdaten (Variante B, siehe DTO-Kapitel).

Methode	Beschreibung
GetBySecretIdHashAsync(hash)	Sucht Zugangsdaten über den Hash der geheimen ID.
GetByUserIdAsync(userId)	Sucht die Zugangsdaten eines bestimmten Benutzers.

OTPRepository

Zweck: Zugriff auf ausgestellte Einmal-Codes.

Methode	Beschreibung
GetByCodeHashAsync(codeHash)	Sucht einen Code über seinen Hash.

Hinweis: AuthService ruft die Methode GetByCodeAsync auf. Im hier vorliegenden Interface IOTPRepository ist nur GetByCodeHashAsync definiert; die Methode muss ergänzt oder der Aufruf angepasst werden.

LevelRepository

Zweck: Zugriff auf Level-Daten.

Methode	Beschreibung
GetByNameAsync(name)	Sucht ein Level über seinen Namen.
GetLevelForExperiencePointsAsync(xp)	Ermittelt das passende Level zu einem XP-Stand.
GetAllOrderedByXpAsync()	Gibt alle Level sortiert nach benötigten XP zurück.

RoomRepository

Zweck: Zugriff auf Räume, inklusive ihrer Aufgaben.

Methode	Beschreibung
GetByNameAsync(name)	Sucht einen Raum über seinen Namen.
GetRoomsByMaxUnlockLevelAsync(maxLevel)	Gibt alle Räume zurück, die bis zu einem bestimmten Level freigeschaltet sind.
GetRoomsByTopicIdAsync(topicId)	Gibt alle Räume eines Themas zurück, inklusive ihrer Aufgaben.
GetByIdWithExercisesAsync(roomId)	Lädt einen Raum inklusive seiner Aufgaben.

TopicRepository

Zweck: Zugriff auf Themen und die Zuordnung von Räumen zu Themen.

Methode	Beschreibung
GetTopicsByRoomIdAsync(roomId)	Gibt alle Themen zurück, die einen bestimmten Raum enthalten.
GetAllWithRoomsAsync()	Gibt alle Themen inklusive ihrer Räume und Aufgaben zurück.
AddRoomToTopicAsync(topicId, roomId)	Ordnet einen bestehenden Raum einem Thema zu.
RemoveRoomFromTopicAsync(topicId, roomId)	Entfernt die Zuordnung eines Raums zu einem Thema.
GetByNameAsync(name)	In der Schnittstelle vorgesehen, aktuell aber nicht umgesetzt (wirft NotImplementedException).

ExerciseRepository

Zweck: Zugriff auf Aufgaben.

Methode	Beschreibung
GetExercisesByRoomIdAsync(roomId)	Gibt alle Aufgaben eines Raums zurück.
GetExercisesByMinXpAsync(minXp)	Gibt alle Aufgaben zurück, die mindestens die angegebene XP-Belohnung bieten, absteigend sortiert.

ExerciseProgressRepository

Zweck: Zugriff auf den Aufgaben-Fortschritt einzelner Benutzer.

Methode	Beschreibung
GetByUserAndExerciseAsync(userId, exerciseId)	Gibt den Fortschrittseintrag zu Benutzer und Aufgabe zurück.
GetAllByUserIdAsync(userId)	Gibt alle Fortschrittseinträge eines Benutzers zurück.
IsExerciseCompletedAsync(userId, exerciseId)	Prüft, ob eine Aufgabe von einem Benutzer bereits abgeschlossen wurde.

RoomProgressRepository

Zweck: Zugriff auf den Raum-Fortschritt einzelner Benutzer.

Methode	Beschreibung
GetByUserAndRoomAsync(userId, roomId)	Gibt den Fortschrittseintrag zu Benutzer und Raum zurück.
GetAllByUserIdAsync(userId)	Gibt alle Raum-Fortschrittseinträge eines Benutzers zurück.
IsRoomCompletedAsync(userId, roomId)	Prüft, ob ein Raum von einem Benutzer bereits abgeschlossen wurde.

6. Controller (API-Endpunkte)

Controller nehmen HTTP-Anfragen entgegen, reichen sie an den passenden Service weiter und geben das Ergebnis als HTTP-Antwort zurück. Alle Controller liegen im Projekt ILSE_Plattform.

AuthController

Basis-Route: api/auth

Zweck: Nimmt die Anmeldung per Einmal-Code entgegen.

HTTP	Pfad	Beschreibung
POST	/login	Tauscht einen Einmal-Code gegen ein JWT-Token; für nicht angemeldete Nutzer frei zugänglich.

OtpController

Basis-Route: api/otp

Zweck: Erstellt neue Einmal-Codes. Nur für Rollen Teacher und Admin zugänglich.

HTTP	Pfad	Beschreibung
POST	/create	Erstellt einen neuen Code für einen Schüler; die aufrufende Benutzer-Id wird aus dem JWT-Token gelesen.

TopicController

Basis-Route: api/topics

Zweck: Verwaltet Themen.

HTTP	Pfad	Beschreibung
POST	/	Legt ein neues Thema an.
GET	/	Gibt alle Themen zurück.
GET	/[{id}]	Gibt ein einzelnes Thema zurück.

RoomController

Basis-Route: api/topics/{topicId}/rooms

Zweck: Gibt die Räume eines Themas zurück.

HTTP	Pfad	Beschreibung
GET	/	Gibt alle Räume eines Themas zurück.
GET	/[{roomId}]	Gibt einen einzelnen Raum zurück.

ExerciseController

Basis-Route: api/topics/{topicId}/rooms/{roomId}/exercises

Zweck: Gibt die Aufgaben eines Raums zurück.

HTTP	Pfad	Beschreibung
GET	/	Gibt alle Aufgaben eines Raums zurück.
GET	/[{exerciseld}]	Gibt eine einzelne Aufgabe zurück.

LevelController

Basis-Route: api/levels

Zweck: Gibt die verfügbaren Level zurück.

HTTP	Pfad	Beschreibung
GET	/	Gibt alle Level zurück.
GET	/[id]	Gibt ein einzelnes Level zurück.

Hinweis: Diese Klasse liegt in der Datei LevelProgress.cs, nicht in einer Datei namens LevelController.cs.

UserProgressController

Basis-Route: api/users/{userId}/progress

Zweck: Gibt den Gesamtfortschritt eines Benutzers zurück.

HTTP	Pfad	Beschreibung
GET	/	Gibt den Fortschritt eines Benutzers zurück.

Hinweis: Nutzt eine Schnittstelle IUserProgressService, die in diesem Export nicht enthalten war.

RoomProgressController

Basis-Route: api/rooms/{roomId}/progress

Zweck: Gibt den Fortschritt eines Benutzers in einem bestimmten Raum zurück.

HTTP	Pfad	Beschreibung
GET	/[userId]	Gibt den Fortschritt eines Benutzers in diesem Raum zurück.

Hinweis: Nutzt eine Schnittstelle IRoomProgressService, die in diesem Export nicht enthalten war.

ExerciseProgressController

Basis-Route: api/exercises/{exerciseld}/progress

Zweck: Gibt den Fortschritt eines Benutzers bei einer bestimmten Aufgabe zurück.

HTTP	Pfad	Beschreibung
GET	/[userId]	Gibt den Fortschritt eines Benutzers bei dieser Aufgabe zurück.

Hinweis: Nutzt eine Schnittstelle IExerciseProgressService, die in diesem Export nicht enthalten war.

7. Hilfsklassen und Infrastruktur

Diese Klassen erledigen technische Teilaufgaben, die von mehreren Services genutzt werden – etwa das sichere Speichern von Passwörtern oder das Erzeugen von Codes und Benutzernamen.

PasswordHasher

Zweck: Speichert Passwörter sicher, indem sie mit PBKDF2 (SHA-256, 100.000 Wiederholungen) und einem zufälligen Salt gehasht werden.

Methode	Beschreibung
Hash(password)	Erzeugt einen neuen Hash und ein zufälliges Salt für ein Passwort.
Verify(password, hash, salt)	Prüft, ob ein eingegebenes Passwort zu einem gespeicherten Hash passt.

OtpHasher

Zweck: Hasht Einmal-Codes, damit sie nicht im Klartext in der Datenbank stehen.

Methode	Beschreibung
Hash(plainValue)	Vereinheitlicht den Code (trimmen, Großbuchstaben) und berechnet einen SHA-256-Hash.

OtpCodeGenerator

Zweck: Erzeugt zufällige, gut lesbare Einmal-Codes.

Methode	Beschreibung
Generate(length)	Erstellt einen kryptografisch zufälligen Code aus einem Alphabet ohne leicht verwechselbare Zeichen (z. B. ohne 0/O, 1/I).

SecretIdGenerator

Zweck: Erzeugt zufällige, sechsstellige geheime Zugangs-IDs.

Methode	Beschreibung
Generate()	Erzeugt eine zufällige, sechsstellige Zahl als String (z. B. "048213").

UsernameFactory

Zweck: Erzeugt zufällige, kindgerechte Benutzernamen aus einer Kombination von Adjektiv und Tier (z. B. "LustigerTiger").

Methode	Beschreibung
CreateRandomUsername (Eigenschaft)	Liefert einen zufälligen Benutzernamen ohne Leerzeichen zwischen Adjektiv und Tier.
GenerateRandomUsername() (statisch)	Liefert einen zufälligen Benutzernamen mit Leerzeichen zwischen Adjektiv und Tier.

Hinweis: Es gibt zwei leicht unterschiedliche Varianten (mit und ohne Leerzeichen) nebeneinander in derselben Klasse.

Guard

Zweck: Kleine Prüfungshilfe, um ungültige Werte früh abzufangen.

Methode	Beschreibung
AgainstEmptyGuid(value, parameterName)	Wirft einen Fehler, wenn eine übergebene Guid leer (Guid.Empty) ist.

UserLevel

Zweck: Enthält eine einfache, rechnerische Levelformel (1000 XP pro Level) als Alternative zu den in der Datenbank hinterlegten Level-Objekten.

Methode	Beschreibung
CalculateLevel(xp)	Berechnet das Level aus den XP ($XP / 1000$).
CalculateExperiencePointsForLevel(level)	Berechnet die für ein Level nötigen XP ($Level \times 1000$).
CalculateExperiencePointsToNextLevel(xp)	Berechnet, wie viele XP bis zum nächsten Level fehlen.

Hinweis: LevelService ermittelt das aktuelle Level stattdessen über die in der Datenbank gespeicherten Level-Objekte. Beide Berechnungswege bestehen aktuell nebeneinander.

8. Datenbank-Anbindung und Programmstart

AppDbContext

Zweck: Der zentrale Datenbankkontext (Entity Framework Core). Verbindet die Fachklassen mit den Datenbanktabellen und baut zusätzlich auf ASP.NET Core Identity auf (für Login-Technik).

Enthaltene Tabellen (DbSets): Topics, Rooms, Exercises, Levels, OneTimePasswords, ExerciseProgresses, RoomProgresses, UserProgresses, Users. Beim Start werden automatisch alle Konfigurationsklassen (siehe unten) aus der Assembly geladen.

8.1 Konfigurationsklassen (EF-Core Mapping)

Für die meisten Fachklassen gibt es eine passende *Configuration-Klasse (z. B. RoomConfiguration, UserConfiguration). Sie legen technische Details für Entity Framework fest: Tabellename, Primärschlüssel und Sonderfälle wie die Umwandlung von Listen in JSON-Text für die Speicherung. Fachlich enthalten sie keine eigene Logik, weshalb sie hier nur als Gruppe aufgeführt werden statt einzeln.

Konfigurationsklasse	Gehört zu
UserConfiguration	User
UserCredentialConfiguration	UserCredential
OneTimePasswordConfiguration	OneTimePassword
TopicConfiguration	Topic
RoomConfiguration	Room
LevelConfiguration	Level
ExerciseConfiguration	Exercise
MultipleChoiceExerciseConfiguration	MultipleChoiceMultiAnswerExercise
DragAndDropExerciseConfiguration	DragAndDropExercise
LinkingExerciseConfiguration	LinkingExercise
ClickableImageExerciseConfiguration	ClickableImageExercise
ClickableAreaConfiguration / DraggableItemConfiguration / DropZoneConfiguration / DragAndDropMappingConfiguration / LinkConfiguration / MCOptionConfiguration	die jeweiligen Aufgaben-Hilfsklassen
UserProgressConfiguration / RoomProgressConfiguration / ExerciseProgressConfiguration	die jeweiligen Fortschrittsklassen

8.2 Programmstart

DependencyInjection (Persistence)

Zweck: Statische Erweiterungsmethode, die alle Persistence-Bestandteile beim Start registriert.

Methode	Beschreibung
AddPersistence(services, configuration, env)	Registriert den Datenbankkontext (SQLite in Entwicklung/Test, PostgreSQL sonst), die Identity-Dienste und alle Repository-Implementierungen.

Program.cs

Zweck: Einstiegspunkt der Web-API. Baut die Anwendung zusammen und startet den Webserver.

Hinweis: Registriert unter anderem: Controller, Swagger (API-Dokumentation im Browser), JWT-Authentifizierung, CORS für den lokalen Frontend-Entwicklungsserver (Port 5173) sowie die Application- und Persistence-Schicht.

9. Anhang: Auffälligkeiten in diesem Codestand

Die folgenden Punkte wurden bei der Code-Analyse festgestellt und sollten vor dem produktiven Einsatz geprüft werden:

- `BaseRepository.DeleteAsync`: Die Bedingung ist vertauscht – aktuell wird der Fehler "nicht gefunden" ausgelöst, wenn der Eintrag gefunden wurde, statt wenn er fehlt.
- `AuthService` ruft `_otpRepo.GetByCodeAsync(...)` auf; `IOTPRepository` definiert in diesem Export nur `GetByCodeHashAsync`.
- Für die Anmeldung existieren zwei parallele DTO-Sätze mit teils gleichen Namen (z. B. zwei `LoginRequest`-Klassen) für zwei unterschiedliche Anmelde-Konzepte.
- `RoomProgress` ist als eigene Klassendatei nicht im Export enthalten; der Aufbau wurde aus Konfiguration und `Repository` abgeleitet.
- `OrderExercise` ist eine leere Platzhalterklasse ohne Attribute – ein noch nicht umgesetzter fünfter Aufgabentyp.
- `ValidateAnswer(...)` ist in allen vier Aufgabentypen nur ein Platzhalter (wirft `NotImplementedException`); die Bewertungslogik der Antworten ist nicht implementiert.
- Die Controller `UserProgressController`, `RoomProgressController` und `ExerciseProgressController` greifen auf Services zu (`IUserProgressService`, `IRoomProgressService`, `IExerciseProgressService`), deren Dateien nicht Teil dieses Exports waren.
- Einige Datei- und Klassennamen enthalten Tippfehler oder weichen leicht voneinander ab (z. B. `RoomControler`, `RoomPrrogressController`, `UserCrendtialRepository`, `ClickableImageExeciseConfiguration`) – ohne Auswirkung auf die Funktion, jedoch relevant für die Suche im Code.